



poeHal

Guía de instalación y uso

CLI

Python

Web

Versión 3.0.0

Switch PoE++ PLANET IGS-4215-8UP2T2S · 192.168.1.103

Revolution Pi SWB0035 · 192.168.1.102

Septiembre 2026

jebi.ai

Qué es poeHal

poeHal controla y monitorea los cubos alimentados por el switch PoE++ PLANET IGS-4215-8UP2T2S. Los cubos son los puertos PoE que les dan energía: **cube1** a **cube5**.

Se usa en **tres modalidades sobre el mismo equipo**: una herramienta de línea de comandos, un paquete de Python y la interfaz web que el propio switch trae de fábrica.

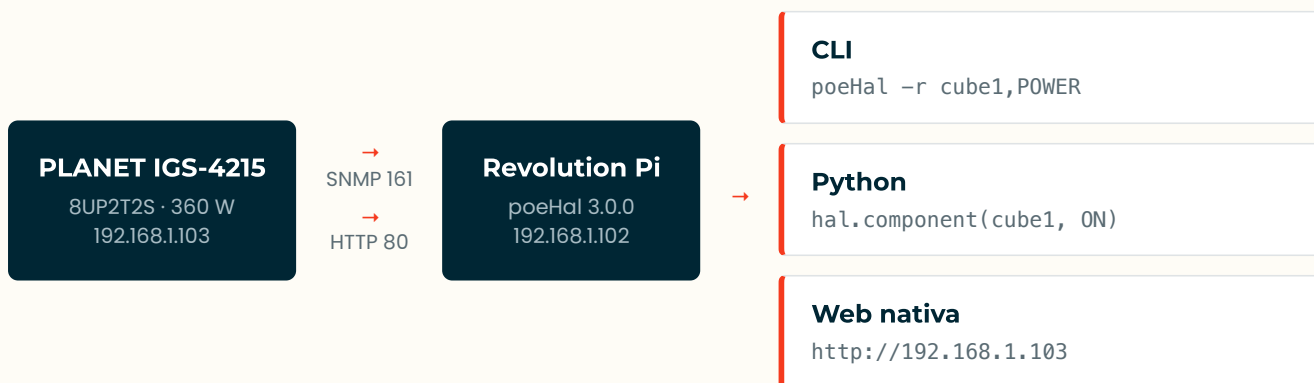
Una diferencia con upsHal. Aquí no hay un proceso intermedio: cada llamada habla directo con el switch. Y la tercera modalidad no es una aplicación nuestra, es la web nativa del fabricante, la misma página que poeHal lee por debajo.

Dos transportes hacia el mismo equipo

Ninguno alcanza por sí solo, así que poeHal abre los dos en cada sesión.

Transporte	Puerto	Qué aporta
SNMP v2c	UDP 161	Potencia agregada del PSE e info del sistema (MIB 1.3.6.1.2.1.105)
HTTP	TCP 80	Datos por cubo y todas las escrituras, leyendo la misma página del switch

Arquitectura



La web nativa no pasa por el Revolution Pi: el navegador habla directo con el switch.

Las tres modalidades

Modalidad	Para qué sirve	Se usa así
CLI	Consulta rápida en terminal, scripts de shell	poeHal -r cube1,POWER
Python	Integrar los cubos en otro sistema o automatización	hal.component(cube1, ON)
Web	Configuración avanzada del switch y diagnóstico visual	http://192.168.1.103

Instalación

Un solo instalador deja operativas las dos modalidades de software. La web nativa no se instala: ya viene en el switch.

REQUISITOS

- Revolution Pi con Raspberry Pi OS y **Python 3.8 a 3.11**
- Acceso de red al switch en `192.168.1.103` , con SNMP habilitado
- Usuario y contraseña de la web del switch

El tope de Python es real. `pysnmp 4.4.12` importa `asyncore` , que fue removido del lenguaje en Python 3.12. El instalador rechaza 3.12 o superior en vez de dejar una instalación que no arranca.

PASO 1. CLONAR E INSTALAR

```
$ git clone https://github.com/ulises-jebi/poeHal.git ~/poeHal
$ cd ~/poeHal
$ sudo ./install.sh
```

PASO 2. CARGAR LAS CREDENCIALES DEL SWITCH

El instalador deja la plantilla; hay que editarla antes del primer uso:

```
$ sudo nano /etc/poeHal/config.ini
```

```
[switch]
host      = 192.168.1.103
snmp_port = 161
community = public
web_user   = jebi
web_pass   = TU_PASSWORD
```

PASO 3. VERIFICAR

```
$ poeHal -r switch,POWER
$ poeHal -r cubes
$ python3 ~/poeTest.py
```

Si **poeHal** no se encuentra, es que `~/local/bin` no está en el PATH. Agregar `export PATH="$HOME/.local/bin:$PATH"` a `~/bashrc` . En cron y en SSH no interactivo conviene la ruta completa: `/home/pi/.local/bin/poeHal` .

Qué deja instalado

El instalador corre siete pasos. El paquete va al home del usuario, no al sistema.

Dónde queda cada cosa

Qué	Dónde	Modalidad
Paquete Python y CLI	<code>~/.local/lib/...</code> y <code>~/.local/bin/poeHal</code>	CLI, Python
Credenciales del switch	<code>/etc/poeHal/config.ini</code>	CLI, Python
Ejemplo ejecutable	<code>~/poeTest.py</code>	Python
Interfaz web	en el switch, nada que instalar	Web

Qué hace cada paso

Paso	Acción
1 y 2	Detecta el usuario destino y un Python entre 3.8 y 3.11
3	Instala snmp y python3-pip del sistema
4	Limpia la instalación antigua en <code>/opt/poeHal</code> y su wrapper
5	Instala el paquete con <code>pip install --user</code> como el usuario, no como root
6	Crea <code>/etc/poeHal/config.ini</code> en modo 600, conservando el existente
7	Copia el ejemplo al home como <code>~/poeTest.py</code>

Por qué `--user` y no `root`. Con `sudo`, un `pip install --user` instalaría en `/root/.local` y el usuario no vería nada. El instalador lo ejecuta como `$SUDO_USER`. Como efecto, poeHal queda disponible para ese usuario, no para cron de sistema.

Desinstalación

```
$ cd ~/poeHal && sudo ./uninstall.sh
```

Quita el paquete y el CLI. **No borra** `/etc/poeHal/config.ini`, porque tiene las credenciales: eso queda a mano con `sudo rm -rf /etc/poeHal`. El `~/poeTest.py` solo se borra si no fue editado.

Reinstalar no pisa la configuración. El paso 6 conserva un `config.ini` existente, así que actualizar es `git pull && sudo ./install.sh` sin volver a cargar credenciales.

01 Línea de comandos

Consulta y control desde la terminal del RevPi o en scripts de shell

Sintaxis

Siempre **dispositivo y parámetro**, separados por coma:

```
$ poeHal -r cube1,STATUS      # Todo el cubo 1
$ poeHal -r cube1,POWER       # Solo la potencia
$ poeHal -r cube1              # Igual que cube1,STATUS
$ poeHal -r switch,POWER      # Consumo total del switch
```

Salida

```
$ poeHal -r cube1,POWER
cube1,POWER = 10.7 W

$ poeHal -r switch,POWER
switch,POWER = 44 W
```

Escritura

```
$ poeHal -w cube3,ON          # Encender
$ poeHal -w cube3,OFF         # Apagar
$ poeHal -w cube3,RESTART     # Off, espera 5 s, on
$ poeHal -w cube3,RESTART,10  # Con espera de 10 s
$ poeHal -w cube1,ON cube5,OFF # Varios cubos a la vez
```

```
$ poeHal -w cube5,OFF
cube5: Enable -> Disable : OK
```

Catálogo, vistas y monitoreo

Comando	Qué hace
<code>poeHal -r components</code>	Lista los dispositivos y todos sus parámetros
<code>poeHal -r cubes</code>	Tabla de los cinco cubos
<code>poeHal -r status</code>	Resumen: potencia, temperaturas y cubos
<code>poeHal -r watch[,seg]</code>	Monitor en vivo, cada 5 segundos por defecto
<code>poeHal -r csv · -r log</code>	Exporta un snapshot, o agrega una línea al log continuo
<code>poeHal help</code>	Ayuda completa y origen de la configuración

La validación ocurre antes de conectar. Un nombre mal escrito responde al instante y una orden con varios cubos no se ejecuta a medias. Las sintaxis viejas avisan el reemplazo: `cube3,1` manda a `cube3,ON`.

02 Paquete Python

Una sola función para integrar los cubos en cualquier automatización

Uso básico

```
import poeHal as hal
from poeHal import cube1, switch, ON, OFF, RESTART, STATUS, POWER

hal.component(cube1, ON)           # True si se confirmó el cambio
hal.component(cube1, OFF)
hal.component(cube1, RESTART, wait=10) # off, espera, on
hal.component(cube1, POWER)         # 10.7
hal.component(cube1, STATUS)        # dict con los 10 parámetros
hal.component(switch, POWER)        # 44
hal.component("cube1", "ON")        # también acepta texto
```

Varios valores del mismo instante

Cada llamada trae una lectura fresca. Con `refresh=False` se reutiliza la anterior, así los cinco cubos son del mismo momento y es una sola consulta al switch:

```
primero = True
for nombre in hal.CUBES:
    print(nombre, hal.component(nombre, STATUS, refresh=primero))
    primero = False
```

Manejo de errores

La librería lanza excepciones, no termina el proceso:

```
try:
    hal.component(cube1, ON)
except hal.ConfigError:      # faltan credenciales o permisos inseguros
except hal.ConnectionFailed: # el switch no responde o rechazó el login
except ValueError:          # dispositivo o parámetro inexistente
```

Utilidades

Llamada	Devuelve
<code>hal.devices()</code>	Los seis dispositivos consultables
<code>hal.parameters(cube1)</code>	Los parámetros válidos para ese dispositivo
<code>hal.describe(POWER)</code>	Etiqueta y unidad
<code>hal.configure(host=...)</code> · <code>hal.reset()</code>	Apunta a otro switch en runtime y fuerza reconexión

Las excepciones heredan de `hal.PoEHalError`. La conexión es perezosa: `import poeHal` no toca la red.

03 Web nativa del switch

La interfaz del fabricante, para configuración avanzada y diagnóstico

Acceso

```
http://192.168.1.103
```

La raíz redirige por JavaScript a `/cgi-bin/dispatcher.cgi?cmd=0`, que es la pantalla de acceso. Se entra con el mismo usuario y contraseña que están en `/etc/poeHal/config.ini`.

Qué ofrece

Es la interfaz completa del equipo, mucho más amplia que poeHal: VLAN, puertos, espejeo, QoS, respaldos de firmware y configuración. Para los cubos, lo relevante vive en la sección PoE.

Sección	Contenido
PoE Configuration	Habilitar cada puerto, prioridad, límite asignado, modo inline y tipo de PD
PoE Status	Corriente, potencia y clase del dispositivo por puerto, en vivo
Power Budget	Presupuesto total, consumo y las dos temperaturas del PSE
PoE Schedule	Encendido y apagado por horario, que poeHal no expone
System	Firmware, hostname, red y respaldo de configuración

Es la misma fuente que lee poeHal

La página PoE (`cmd=9216`) publica su estado como arreglos de JavaScript dentro del HTML. poeHal los extrae de ahí, y escribe con un POST a `cmd=9217`. En otras palabras, la web y poeHal son dos clientes del mismo endpoint, con los mismos datos.

No escribir por los dos lados a la vez. El CGI del switch solo acepta la configuración de **los ocho puertos completa**: cada cambio reenvía todo el formulario. Si la web tiene la página PoE abierta con cambios sin guardar y poeHal escribe en ese momento, el que guarde último pisa al otro. Para automatizar, usar poeHal; para configurar a mano, cerrar la sesión web al terminar.

Sobre la red. La interfaz usa HTTP plano y SNMP v2c manda el community en texto claro. Las credenciales viajan sin cifrar, así que este equipo debe permanecer en un segmento OT aislado.

Referencia

Los mismos dispositivos y parámetros en el CLI y en Python. En el equipo se listan con `poeHal -r components` o `hal.parameters()`.

Hay seis dispositivos: `cube1` a `cube5`, que son los cubos en los puertos PoE 1 a 5, y `switch`, que agrupa los datos del equipo completo.

Parámetros de un cubo

Parámetro	Devuelve	Unidad
STATUS	Los diez parámetros en un dict	
ENABLED	Si el puerto está habilitado	bool
DELIVERING	Si hay un dispositivo consumiendo	bool
POWER	Potencia entregada	W
CURRENT	Corriente entregada	mA
MAXPOWER	Máximo asignado al puerto	W
PRIORITY	Critical, High o Low	
PDCLASS	Clase del dispositivo alimentado	
PDTYPE	Standard, Legacy o Force	
INLINE	End-Span, Mid-Span o BT	
EXTEND	Modo extendido	
ON · OFF · RESTART	Escriben. Devuelven si se confirmó el cambio	bool

Parámetros del switch

Parámetro	Devuelve	Unidad
STATUS	Los nueve parámetros en un dict	
PSE	ON, OFF o FAULTY	
NOMINAL	Potencia nominal del PSE	W
CONSUMED · POWER	Consumo total, el mismo valor	W
PERCENT	Uso sobre la nominal	%
BUDGET	Presupuesto configurado	W
TEMPERATURE	Las dos sondas del PSE, como lista	°C
DESCR · NAME · UPTIME	Descripción, hostname y tiempo encendido	

Cinco cubos, ocho puertos. El switch tiene ocho puertos PoE, pero solo los cinco primeros son cubos y son los únicos expuestos. Por dentro cada escritura reenvía la configuración de los ocho, porque así lo exige el CGI: recortarla corrompería 6 a 8.

Problemas comunes

Síntoma	Causa y solución
El comando poeHal no existe	<code>~/ .local/bin</code> no está en el PATH. Agregar <code>export PATH="\$HOME/.local/bin:\$PATH"</code> a <code>~/ .bashrc</code> . En cron y SSH no interactivo, usar la ruta completa.
poeHal -r cubes no imprime nada	Es un fallo, no un éxito: el login web fue rechazado. Verificar <code>web_user</code> y <code>web_pass</code> en el config, y probar esas mismas credenciales en <code>http://192.168.1.103</code> .
<i>Permisos inseguros al arrancar</i>	El config.ini es legible por otros usuarios. El mensaje trae el <code>chmod 600</code> exacto.
<i>No hay credenciales configuradas</i>	No se encontró ningún config. El propio error lista las dos formas de cargarlas, por archivo o por variables <code>POEHAL_*</code> .
SNMP no responde	Problema de red o SNMP deshabilitado. Comprobar con <code>snmpget -v2c -c public 192.168.1.103 1.3.6.1.2.1.1.1.0</code> .
Falla al instalar en Python 3.12+	No es un error de configuración: <code>pysnmp 4.4.12</code> necesita <code>asyncore</code> , removido en 3.12. Instalar <code>python3.11</code> .
<code>port3</code> o <code>cube3,1</code> dan error	Sintaxis anterior a la 3.0. El mensaje indica el reemplazo exacto: <code>cube3</code> y <code>cube3,0N</code> .
Un cubo quedó apagado tras una prueba	<code>examples/minimo.py</code> termina encendiendo. Si se interrumpió a la mitad, restaurar con <code>poeHal -w cube5,0N</code> .

Comandos de diagnóstico

```
$ poeHal help                # origen de la configuración
$ poeHal -r components       # catálogo completo
$ poeHal -r switch,STATUS    # estado agregado
$ ping -c 2 192.168.1.103    # red hacia el switch
$ snmpwalk -v2c -c public -Oqv 192.168.1.103 1.3.6.1.2.1.105.1.1.1.3
```

Verificar una escritura de forma independiente

Cada escritura reenvía los ocho puertos, así que conviene comprobar que solo cambió el cubo que se tocó:

```
$ poeHal -r cubes > /tmp/antes.txt
$ poeHal -w cube5,0FF && poeHal -w cube5,0N
$ poeHal -r cubes > /tmp/despues.txt && diff /tmp/antes.txt /tmp/despues.txt
```

El ejemplo `examples/prueba_paquete.py --cube 5` hace esa misma comprobación y avisa si algún otro cubo cambió.

Repositorio. github.com/ulises-jebi/poeHal · Pruebas sin switch ni red: `python3 tests/test_config.py`, 42 comprobaciones.